

Git State Management and Recovery Discipline

Resilient Operational Patterns for Google Antigravity 2.0 CLI (agy) Sessions

Context and User Inquiry

Original Query

“If my Ubuntu 24.04 VM were unstable I would consider using tmux, but it has not crashed since LLMs helped me make it stable. So I will not likely need tmux. I only worry about leaving the repo in unknown state if the system reboots during agy sessions that are saving work to the repo. Is there a git discipline that makes it easier to return to known good state after a disconnect or something else that leaves the repo in an unknown state? Can we versions or release numbers or tags or something? I am not very familiar with git let alone a discipline that might be useful here.”

1 Introduction: Branches vs. Tags and Version Numbers

When managing autonomous code-generation workflows using agentic command-line tools like **agy**, unexpected interruptions—such as an operating system reboot, network drop, or virtual machine shutdown—can occur in the middle of multi-file write operations. This runs the risk of leaving the repository in a corrupted, inconsistent, or untracked state.

A common intuition is to apply semantic versioning numbers or Git tags before running an agent. However:

- **Git Tags and Version Numbers** represent immutable milestones intended to mark permanent releases (e.g., `v1.0.0`, `v1.2.3-rc1`) in project history. They are metadata pointers created to look backward at completed, verified work.
- **Ephemeral Feature Branches** provide dynamic, disposable sandboxes designed specifically for in-flight, exploratory, or automated mutations.

Using a dedicated branching discipline rather than version tags eliminates the risk of an unknown repository state, enabling an instantaneous rollback to a known good state with a single command.

2 The Three-Pillar Git Discipline

2.1 Pillar 1: Always Isolate Agent Work in a Disposable Branch

Never invoke **agy** directly on a trunk branch such as `main` or `master`. Prior to starting an agent session, branch off into a dedicated sandbox prefixed with a clear namespace (e.g., `agy/`):

```
# Create and switch to a disposable agent branch
git checkout -b agy/session-task-name
```

If an abrupt reboot occurs mid-session, the integrity of `main` remains intact. If the session output is determined to be malformed or unrecoverable, the entire branch can be discarded:

```
# Abandon the corrupted session and return to safety
git checkout main
git branch -D agy/session-task-name
```

2.2 Pillar 2: Verify a Pristine Starting State

An autonomous agent that starts in a repository containing dirty unstaged changes or loose files will interweave new modifications with existing work. Before running `agy`, ensure that the working tree is clean:

```
# Check working directory state
git status

# If uncommitted edits exist, stow them away cleanly
git stash -u # -u includes untracked files
```

2.3 Pillar 3: The Deterministic One-Command Recovery Hatch

If the system restarts unexpectedly and you return to find half-written files, partial patches, or syntax errors, there is no need to manually audit what the agent touched.

Because the session started from a known clean commit on the branch, the repository can be returned to that exact state instantly:

```
# 1. Revert all modified tracked files to the initial commit
git reset --hard HEAD

# 2. Remove all newly created untracked files and directories
git clean -fd
```

Recovery Hatch Summary

The Golden Guarantee: Running `git reset -hard HEAD && git clean -fd` completely purges every uncommitted file alteration and newly created artifact, resetting the workspace to the exact byte-level state of the last recorded commit.

3 Standard Operating Procedure (SOP) Lifecycle

The recommended operational cycle for running `agy` sessions without state contamination follows a structured four-stage pattern:

1. **Pre-Flight (Baseline):** Ensure your base branch is up to date and clean.

```
git checkout main
git pull origin main
git status # Must show 'working tree clean'
```

2. **Branch & Launch:** Isolate the execution context.

```
git checkout -b agy/refactor-pipeline
agy
```

3. **Post-Session Triage (Post-Flight):**

- **Scenario A: Interrupted / Corrupted.** Run the recovery hatch:

```
git reset --hard HEAD
git clean -fd
```

- **Scenario B: Successful Run.** Inspect diffs, test, and commit:

```
git diff
git add -A
git commit -m "feat(pipeline): automated refactor via agy"
```

4. **Integration or Teardown:**

- If accepted: Merge back into `main` via fast-forward or squash merge:

```
git checkout main
git merge agy/refactor-pipeline
git branch -d agy/refactor-pipeline
```

- If rejected: Force delete the branch without touching `main`:

```
git checkout main
git branch -D agy/refactor-pipeline
```

4 Automating the Workflow with a BASH Helper

To avoid repetitive manual branch creation and pre-flight checks, add the following helper function to your `~/.bashrc`:

```
# Safe wrapper for Google Antigravity CLI sessions
agy_sandbox() {
    local task_name="${1:-scratch}"
    local branch_name="agy/${task_name}-${date +%Y%m%d-%H%M%S}"

    # Ensure repository is clean
    if ! git diff-index --quiet HEAD -- 2>/dev/null; then
        echo "Error: Working directory has uncommitted changes. Stash or
            commit first."
        return 1
    fi

    echo "Creating sandbox branch: ${branch_name}"
    git checkout -b "${branch_name}" || return 1
}
```

```

echo "Launching Antigravity CLI..."
agy

echo ""
echo "Session completed or exited."
echo "Current branch: ${branch_name}"
echo "To keep work:      git add -A && git commit -m '...'"
echo "To reset state:    git reset --hard HEAD && git clean -fd"
echo "To abandon:        git checkout main && git branch -D
                        ${branch_name}"
}

```

5 Summary Reference Table

Goal	Git Command	Operational Effect
Isolate agent session	<code>git checkout -b agy/<task></code>	Prevents pollution of <code>main</code>
Stash local work	<code>git stash -u</code>	Preserves prior uncommitted work
Audit agent changes	<code>git status</code> / <code>git diff</code>	Reviews modified and new files
Revert tracked edits	<code>git reset -hard HEAD</code>	Restores tracked files to base
Purge new artifacts	<code>git clean -fd</code>	Deletes unversioned files/dirs
Abandon entire attempt	<code>git checkout main && git branch -D <branch></code>	Leaves zero trace of session